

Documentation insert - assigning the TagProtect access password

Drop-in text for the REST and MQTT documentation of `PUT /cloud/impinjGen2X / set_impinjGen2X`.

Why it is needed: both documents already state the prerequisite -

"The target tag must have a valid 32-bit Access Password configured before using Protected Mode operations."

- but neither says where the password lives or how to configure one. The procedure below is verified working and fills only that gap.

Two changes are proposed:

#	Where	Change
A	## 3. Before You Begin, TagProtect password row	Replace one table row
B	## 5. Choosing TagProtect Actions, at the end	Add a new subsection

Everything else in both documents stays as it is.

Change A - one row in ## 3. Before You Begin

Current row:

What You Need	Details
TagProtect password	An 8-character hex string (32-bit) is required for all TagProtect actions.

Replace with:

What You Need	Details
TagProtect password	An 8-character hex string (32-bit) is required for all TagProtect actions, and it must already be stored on the target tag. The tag's access password occupies words 2-3 of the RESERVED memory bank. Read it, or assign one, using a <code>set_mode</code> access operation before configuring TagProtect - see Assigning the Access Password. Many tags ship with a password already set; read it first and use that value.

Change B - new subsection at the end of ## 5. Choosing TagProtect Actions

Insert the following after the existing `> Important:` note.

Assigning the Access Password

Every TagProtect action authenticates against the tag's 32-bit access password, which must already be stored on the tag. The access password occupies words 2-3 of the Gen2 `RESERVED` memory bank:

Words	Contents
0-1	Kill password
2-3	Access password

Reading and writing this bank is done with a `set_mode` access operation, not through `impinjGen2X`. Because access operations execute when the tag is singulated, each read or write must be followed by an inventory.

Step 1 - Read the current password

Many tags are supplied with an access password already programmed. Read words 0-3 before assigning a new one.

REST - `PUT /cloud/mode`

```
{
  "type": "CUSTOM",
  "antennas": [1],
  "transmitPower": 30,
  "filter": {
    "value": "e28011b0a5050076c4d7530a",
    "match": "prefix",
    "operation": "include"
  },
  "accesses": [
    {
      "type": "READ",
      "config": {
        "membank": "RESERVED",
        "wordPointer": 0,
        "wordCount": 4
      }
    }
  ]
}
```

MQTT - `set_mode`

```
{
  "command": "set_mode",
  "payload": {
    "type": "CUSTOM",
    "antennas": [1],
    "transmitPower": 30,
    "filter": {
      "value": "e28011b0a5050076c4d7530a",
      "match": "prefix",
      "operation": "include"
    },
    "accesses": [
      {
        "type": "READ",
        "config": {
          "membank": "RESERVED",
          "wordPointer": 0,
          "wordCount": 4
        }
      }
    ]
  }
}
```

Run an inventory to execute the read:

Action	REST	MQTT
Start	PUT /cloud/start with {}	{"command": "start", "payload": {}}
Stop	PUT /cloud/stop	{"command": "stop", "payload": {}}

The result arrives in `accessResults` in the tag data stream as 16 hexadecimal characters covering words 0-3:

```
{"data": {"idHex": "e28011b0a5050076c4d7530a",
  "accessResults": ["1234876512348765"]}}
```

Characters	Field
1-8	Kill password (words 0-1)
9-16	Access password (words 2-3)

Value read	Action
0000000000000000	No password set - assign one in step 2
Any other value	A password is already set - use the last 8 characters and skip step 2

Step 2 - Assign a password

Write 8 hexadecimal characters (2 words = 32 bits) to **wordPointer: 2**.

REST - **PUT** /cloud/mode

```
{
  "type": "CUSTOM",
  "antennas": [1],
  "transmitPower": 30,
  "filter": {
    "value": "e28011b0a5050076c4d7530a",
    "match": "prefix",
    "operation": "include"
  },
  "accesses": [
    {
      "type": "WRITE",
      "config": {
        "membank": "RESERVED",
        "wordPointer": 2,
        "data": "A1B2C3D4"
      }
    }
  ]
}
```

MQTT - **set_mode**

```
{
  "command": "set_mode",
  "payload": {
    "type": "CUSTOM",
    "antennas": [1],
    "transmitPower": 30,
    "filter": {
      "value": "e28011b0a5050076c4d7530a",
      "match": "prefix",
      "operation": "include"
    },
    "accesses": [
      {
        "type": "WRITE",
        "config": {
          "membank": "RESERVED",
          "wordPointer": 2,
          "data": "A1B2C3D4"
        }
      }
    ]
  }
}
```

Run an inventory as in step 1. Each singulation produces a result:

```
{"data": {"idHex": "e28011b0a5050076c4d7530a", "accessResults": ["SUCCESS"]}}
```

Intermittent **CRC error in tag response** or **Tag did not respond** entries are normal RF behaviour when the tag is at the edge of range; at least one **SUCCESS** confirms the write.

Include a `filter`. Without one, the write is applied to every tag in the field. Use the target EPC with **"match": "prefix"** to address a single tag.

Do not use `00000000`. A zero access password means "no password set" on a Gen2 tag, so the tag cannot be protected. The request is accepted without error.

Step 3 - Verify

Repeat the step 1 read. The last 8 characters of **accessResults** must match the password that was written:

```
{"data": {"idHex": "e28011b0a5050076c4d7530a",  
  "accessResults": ["00000000a1b2c3d4"]}}
```

Verification is required because TagProtect requests are accepted whether or not the supplied password matches the tag. A password mismatch returns **200 OK** and leaves the tag unprotected.

Step 4 - Configure TagProtect

Use the same password in the TagProtect request.

REST - PUT /cloud/impinjGen2X

```
{  
  "tagProtect": {  
    "action": "enableTagProtection",  
    "password": "A1B2C3D4",  
    "tagID": "e28011b0a5050076c4d7530a"  
  }  
}
```

MQTT - set_impinjGen2X

```
{  
  "command": "set_impinjGen2X",  
  "payload": {  
    "tagProtect": {  
      "action": "enableTagProtection",  
      "password": "A1B2C3D4",  
      "tagID": "e28011b0a5050076c4d7530a"  
    }  
  }  
}
```

Apply it on the next inventory start:

REST	MQTT
PUT /cloud/start with {"applyImpinjGen2X": true}	{"command": "start", "payload": {"applyImpinjGen2X": true}}

Password requirements

Requirement	Notes
Length	Exactly 8 hexadecimal characters (32-bit)
Characters	0-9 and A-F only
Case	Not significant - a1b2c3d4 and A1B2C3D4 are equivalent
Reserved value	00000000 must not be used; it denotes "no password" on a Gen2 tag

Changing or clearing a password

Writing to words 2-3 again replaces the previous value; there is no separate change operation.

Unprotect the tag with `disableTagProtection` before changing or clearing its access password. If the password is changed while the tag is protected, the value required to unprotect it can no longer be supplied and the tag remains unreadable.

Verification

Every payload above was executed against live readers:

Readers	FXR60 reader application 5.0.7; FXR90 reader application 5.0.4
Tags	Impinj Monza 4i (MDID 0x801, TMN 0x1B0)
Protect confirmed	Target tag visible 5/5 inventories before protection, 0/5 after, with a control tag visible 5/5 throughout
Restored	<code>disableTagProtection</code> returned the tag to 5/5 visibility

Supporting detail, including the measurement and raw captures: `tagprotect-CONFIRMED-monza4i/TAGPROTECT_CONFIRMED.md`